Colecțiile din .NET în proiectul tău

Ștefan · 6 exerciții în `stefan-academy-vanilla-csharp` · fără LINQ (vine în partea 3)

Ai învățat fiecare colecție în `collections-dotnet`; acum le pui în proiectul tău, acolo unde codul chiar le cere. Fiecare exercițiu are: **Unde** (fișierul și liniile de azi, din commit-ul `c2b024b`), **De ce** (rândul din tabelul „Ce colecție alegi” din SCHEMA.pdf), **Cerința** și **Gata când**, cu ieșirea exactă.
Înainte: cele 3 🔴 din runda 12 (CODE_REVIEW.pdf). La exercițiile 3, 4 și 6 ieși din meniul Cursuri ca să ajungi la Statistici, deci `case 0` trebuie să meargă.
Reguli: lucrezi în clasele existente, în stilul lor (View → Service → Repository) · fără LINQ · un exercițiu per commit · `dotnet build` fără erori · numele noi de metode sunt cele din cerință.
Datele: fiecare „Gata când” pornește de la fișierele din `Data/` așa cum sunt în git. Le aduci înapoi cu `git checkout -- Data/books.txt Data/courses.txt Data/enrolments.txt Data/users.txt`.
În ieșiri: textul **îngroșat** e ce tastezi tu. Liniile de meniu nu sunt scrise; contează doar liniile arătate.

Partea 1 — Fundația: căutarea după cheie

1. `Repository<T>` găsește după `Id` fără să parcurgă lista

Unde: `Common/Repository.cs:5` și `:35–45`

```
private readonly List<T> items = new();
...
public T FindById(Guid id)
{
    foreach (T item in items)
    {
        if (item.Id == id)
        {
            return item;
        }
    }
    return null;
}
```

De ce `Dictionary<Guid, T>`: `FindById` e chemat de fiecare `Update…` și `Delete…` din cele 4 servicii și de `GetCourseListByCourseId`. Fiecare apel trece prin toată lista până dă de `Id`. SCHEMA, „Ce colecție alegi”: *„Caut ceva după o cheie (nume, cod)”* → `Dictionary<K, V>`, `TryGetValue`.

Cerința:

1. În `Repository<T>`, lângă `items`, câmpul `private readonly Dictionary<Guid, T> itemsById`. `items` rămâne: `Save` scrie în ordinea din fișier, iar repository-urile copil parcurg `Items`.
2. `Read`, `Add` și `Remove` țin cele două colecții la fel: un element e în amândouă sau în niciuna.
3. `FindById` își păstrează semnătura și întoarce tot `null` pentru un `Id` care nu există, dar fără buclă.
4. Dacă la `Read` o linie are un `Id` deja citit, arunci `DuplicateException(typeof(T).Name, id.ToString())`. Azi a doua linie intră în listă, iar `FindById` o găsește doar pe prima.

Gata când: ștergi o carte și totul arată ca înainte. Scenariul: student Stefan Scarlat, CARTI → 4 → `Cartea1`, apoi 1:

```
Tastati numele cartii pe care doriti sa o stergeti: Cartea1
NotificatorTest Book removed successfully
Carte stearsa cu succes
...
Nume carte: Alchimistul , ID carte: 0856a079-9b10-4a52-ac61-39efa8ee63f5 …
Nume carte: Alchimistul , ID carte: 99e99806-3927-437b-bd6b-74973a26f799 …
```

Apoi copiezi prima linie din `Data/courses.txt` pe un rând nou, la sfârșitul fișierului și te loghezi tot ca Stefan Scarlat:

```
Numele: Scarlat
Prenumele: Stefan
Course "db7a391a-ee78-4ae6-a3fb-a8c99a27fbf1" e deja in lista.
```

> **De verificat:** constructorul `Course(string name, string department)` (`Courses/Models/Course.cs:12`), folosit de `CourseMapper.ToCourse`. Ce `Id` primește un curs creat așa? Ce face `itemsById` la al doilea curs creat din aplicație? Scrie răspunsul în `RASPUNSURI.txt`.

2. Fabrica potrivită, după tipul de pe linie

Unde: `Users/Mappers/UserTextMapper.cs:9` și `:41–47`

```
private IUserFactory[] factories = new IUserFactory[] {
    new StudentFactory(), new TeacherFactory(), new AdminFactory() };
...
for (int i = 0; i < factories.Length; i++)
{
    if (factories[i].Type == cuv[0])
    {
        return factories[i].Create(cuv);
    }
}
```

De ce `Dictionary<string, IUserFactory>`: pentru fiecare linie din `users.txt` cauți fabrica după un cod (`"STUDENT"`, `"TEACHER"`, `"ADMIN"`). Codul e chiar cheia, iar fiecare fabrică și-o spune singură prin `Type`.
SCHEMA: *„Caut ceva după o cheie (nume, cod)”* → `Dictionary<K, V>`.

Cerința:

1. `factories` devine `Dictionary<string, IUserFactory>`, cu cheia = `Type`-ul fabricii.
2. Îl umpli în constructorul `public UserTextMapper()`, din aceleași trei fabrici. Codurile `"STUDENT"`, `"TEACHER"`, `"ADMIN"` nu sunt scrise de mână la umplerea dicționarului: le iei din `Type`-ul fabricilor. `ToText` rămâne neschimbat.
3. `FromText` găsește fabrica fără buclă. Pentru un tip necunoscut aruncă aceeași excepție ca azi.

Gata când: login-ul ca Stefan Scarlat afișează `Logat cu succes!`, ca înainte. Apoi adaugi pe un rând nou, la sfârșitul lui `Data/users.txt`, linia `PROFESOR,11111111-1111-1111-1111-111111111111,Ion,Pop,ion.pop@gmail.com,40` și pornești. Prima linie e aceeași ca înainte de schimbare:

```
Unhandled exception. System.ArgumentException: Unknown user type: PROFESOR
```

Partea 2 — Statisticile, dintr-o singură trecere

3. Cursul cu cei mai mulți studenți, cu un numărător

Unde: `ViewStudent.cs:301–303` → `Enrolments/Repositories/EnrolmentRepository.cs:69–81`

```
foreach (Course c in courses)
{
    int aux = enrolmentService.GetStudentsCountForCourseId(c.Id);
...
public int StudentsCountForCourseId(Guid courseId)
{
    int index = 0;
    foreach (Enrolment enr in Items)
    ...
```

De ce `Dictionary<Guid, int>`: pentru fiecare curs parcurgi din nou toate înscrierile: 6 cursuri × 8 înscrieri = 48 de pași, ca să numeri 8 lucruri. Cu un numărător (TEORIE §5) treci o singură dată prin înscrieri. SCHEMA: *„Caut ceva după o cheie”* → `Dictionary<K, V>`; cheia e `CourseId`, valoarea e câți studenți are.

Cerința:

1. În `EnrolmentRepository`: `public Dictionary<Guid, int> StudentsCountPerCourse()`, într-o singură trecere prin `Items`. În `EnrolmentService`: `public Dictionary<Guid, int> GetStudentsCountPerCourse()`.
2. `CursTopStudenti` cere dicționarul o singură dată, înainte de `foreach`. Un curs fără nicio înscriere nu e în dicționar și are 0 studenți, fără excepție.
3. La egalitate câștigă tot primul curs din fișier, ca azi. `StudentsCountForCourseId` și `GetStudentsCountForCourseId` nu mai sunt folosite: le scoți.

Gata când: STATISTICI → 1, cu datele din git:

```
Algebra Matematica cu 2 studenti
```

După CURSURI → 2 → `Algebra` (te înscrii), înapoi la STATISTICI → 1:

```
Ati fost inscris cu succes la curs!
...
Algebra Matematica cu 3 studenti
```

Cu `Data/enrolments.txt` gol (0 octeți):

```
Analiza Matematica cu 0 studenti
```

4. Cursurile la care te mai poți înscrie

Unde: meniul `Cursuri()`, `ViewStudent.cs:100–103`, lângă `Enrolments/Repositories/EnrolmentRepository.cs:54–67`

```
public List<Guid> GetCourseIdListByStudentId(Guid studentId)
{
    List<Guid> courses = new();

    foreach (Enrolment enr in Items)
    {
        if (enr.StudentId == studentId)
        {
            courses.Add(enr.CourseId);
        ...
```

De ce `HashSet<Guid>`: ca să alegi un curs la înscriere, vrei să vezi doar cursurile la care nu ești deja. Pentru fiecare curs întrebi „e printre ale mele?”. Pe o listă, `Contains` o parcurge toată de fiecare dată. SCHEMA: *„Am văzut deja asta?” / „fără dubluri”* → `HashSet<T>`, `Contains`.

Cerința:

1. În `EnrolmentRepository`: `public HashSet<Guid> CourseIdsForStudent(Guid studentId)`. În `EnrolmentService`: `public HashSet<Guid> GetCourseIdsForStudent(Guid studentId)`. `GetCourseIdListByStudentId` rămâne cum e: `AfisareCursuri` îl folosește.
2. În meniul `Cursuri()`, după tasta 3, rândul exact `Apasati tasta 4 pentru a vedea cursurile la care va puteti inscrie`, legat de metoda nouă `public void AfisareCursuriDisponibile()` din `ViewStudent`.
3. Afișează cursurile la care studentul logat nu e înscris, în ordinea din `courses.txt`, cu același format ca `AfisareCursuri`. Dacă nu rămâne niciunul: `Sunteti inscris la toate cursurile`.

Gata când: Stefan Scarlat, CURSURI → 4:

```
nume: Algebra, departament: Matematica
nume: Geometrie, departament: Matematica
nume: Optica, departament: Fizica
nume: Mecanica, departament: Fizica
```

După ce te înscrii (tasta 2) la toate patru, CURSURI → 4:

```
Sunteti inscris la toate cursurile
```

Partea 3 — Rapoarte în ordine

5. Cursurile, pe departamente

Unde: meniul `Statistici()`, `ViewStudent.cs:136–141`, și `Courses/Repositories/CourseRepository.cs:25–28`

```
switch (tasta)
{
    case 0: return;
    case 1: CursTopStudenti(); break;
    default: InputGresit(); break;
}
...
public List<Course> Courses()
{
    return Items;
}
```

De ce `SortedDictionary<string, List<Course>>`: grupezi după departament (cheia) și vrei departamentele în ordine alfabetică, fără să sortezi tu la final. În fiecare grupă, cursurile rămân în ordinea din fișier, deci valoarea e o listă. SCHEMA: *„Caut după cheie și vreau cheile în ordine”* → `SortedDictionary<K, V>`; `foreach` dă cheile crescător.

Cerința:

1. În `CourseRepository`: `public SortedDictionary<string, List<Course>> CoursesByDepartment()`. În `CourseService`: `public SortedDictionary<string, List<Course>> GetCoursesByDepartment()`.
2. În `Statistici()`, după tasta 1, rândul exact `Apasati tasta 2 pentru a vedea cursurile pe departamente`, legat de `public void CursuriPeDepartamente()` din `ViewStudent`.
3. Un rând per departament: `Departament: Curs1, Curs2, ...`.

Gata când: STATISTICI → 2, cu datele din git:

```
Fizica: Optica, Mecanica
Informatica: Algoritmica
Matematica: Analiza, Algebra, Geometrie
```

Provocare

6. Câți studenți are fiecare departament

Unde: tot `Statistici()`; pornești de la exercițiile 3 și 5

De ce `SortedDictionary<string, HashSet<Guid>>`: un student înscris la Analiza și la Algebra e un singur student al departamentului Matematica, nu doi. Dacă aduni numerele din exercițiul 3 pe departament, îl numeri de două ori. Așa că, pentru fiecare departament, strângi studenții într-o mulțime și abia apoi numeri. SCHEMA: *„fără dubluri”* → `HashSet<T>`, `UnionWith`; plus *„cheile în ordine”* → `SortedDictionary<K, V>`.

Cerința:

1. În `EnrolmentRepository`: `public HashSet<Guid> StudentIdsForCourse(Guid courseId)`. În `EnrolmentService`: `public HashSet<Guid> GetStudentIdsForCourse(Guid courseId)`.
2. În `Statistici()`, după tasta 2, rândul exact `Apasati tasta 3 pentru a vedea cati studenti are fiecare departament`, legat de `public void StudentiPeDepartamente()` din `ViewStudent`.
3. Departamentul → mulțimea studenților lui. Mulțimea unui curs intră în cea a departamentului cu `UnionWith`. Un rând per departament, în ordine alfabetică: `Departament: N studenti`. Un departament fără înscrieri apare cu 0.

Gata când: STATISTICI → 3, cu datele din git:

```
Fizica: 3 studenti
Informatica: 1 studenti
Matematica: 4 studenti
```

Te înscrii ca Stefan Scarlat la `Algebra`. Apoi STATISTICI → 1 și → 3:

```
Algebra Matematica cu 3 studenti
...
Fizica: 3 studenti
Informatica: 1 studenti
Matematica: 4 studenti
```

Algebra a crescut, Matematica nu. În `RASPUNSURI.txt`: de ce?